

INDEX

SN	TOPIC	PAGE	REMARKS
1.	WAP TO USE ALL CONSTRUCTOR IN C++ CLASS	1	
2.	WAP TO COUNT THE NUMBER OF OBJECT CREATED USING STATIC MEMBER	2	
3.	WAP TO OVERLOAD PRE AND POST INCREMENT OPERATOR	3	
4.	WAP TO OVERLOAD BINARY OPERATOR (+) AND (*).	4	
5.	WAP TO SHOW ALL CONVERSION (Basic to class), (class to basic) and (class to class)	5	
6.	WAP TO SHOW AN EXAMPLE OF INHERITANCE	6	
7.	WAP TO GIVE EXAMPLE OF POLYMORPHISM OF CLASS SHAPE, RECTANGLE AND TRIANGLE	8	
8.	WAP TO HANDLE EXCEPTION FOR INVALID AGE INPUT.	10	
9.	WAP TO USE TEMPLATE T AND U TO FIND SMALLER NUMBER BETWEEN T AND U.	11	
10.	WAP TO READ A FILE "data.txt" AND DISPLAY ALL THE CONTENT.	12.	

I. WAP TO USE ALL CONSTRUCTOR IN C++ CLASS.

```
#include "iostream"

class Rect {
private:
    float L;
    float W;
public:
    Rect(): L(W(0)) {}
    Rect(float a, float b): L(a), W(b) {}
    Rect(Rect R): L(R.L), W(R.W) {}

public:
    void Display() {
        std::cout << "Length = " << L << std::endl;
        std::cout << "Width = " << W << std::endl;
    }
};

int main() {
    float a=10.0f, b=20.0f, c=13.3f, d=15.0f;
    Rect R1, R2(a,b), R3(c,d);
    Rect R4(R3);
    R1.Display();
    R2.Display();
    R4.Display();
    return 0;
}
```

2. WAP TO COUNT THE NUMBER OF OBJECT CREATED USING
STATIC MEMBER

```
#include <iostream>
class Count {
    static int member;
public:
    Count() { ++member; }
    static inline int Get() { return member; }
};
int Count::member = 0;
int main() {
    Count C1, C2, C3;
    int count;
    count = Count::Get();
    std::cout << "(\n num of obj. created) = " << count << std::endl;
    return 0;
}
```

3. WAP TO OVERLOAD PRE AND POST INCREMENT OPERATOR.

```
#include <iostream>

class Uload {
private:
    int I;
public:
    Uload(int a = 0): I(a) {}
    void operator++() {
        ++I;
    }
    void operator++(int) {
        I++;
    }
    void Display() {
        std::cout << "I = " << I << std::endl;
    }
};

int main() {
    Uload U1(5), U2(8);
    U1++;
    ++U2;
    U1.Display();
    U2.Display();
    return 0;
}
```

4. WAP TO OVERLOAD BINARY OPERATOR(+) AND (*).

```
#include <iostream>
```

```
class Rect{
```

```
    float L;
```

```
    float W;
```

```
public:
```

```
    Rect(): L(W(0)){};
```

```
    void Getdata(){
```

```
        std::cout << "\n Enter length & width" << std::endl;
```

```
        std::cin >> L >> W;
```

```
    }
```

```
    void operator+(Rect B){
```

```
        float X, Y;
```

```
        X = L + B.L;
```

```
        Y = W + B.W;
```

```
        std::cout << "New Length = " << X << " " << "New width = " << Y << std::cout;
```

```
    }
```

```
    void operator*(Rect B){
```

```
        float X;
```

```
        X = L * W + B.L * B.W;
```

```
        std::cout << "\n combined Area = " << X << std::endl;
```

```
    }
```

```
};
```

```
int main(){
```

```
    Rect R; Rect R1;
```

```
    R.Getdata();
```

```
    R1.Getdata();
```

```
    R + R1; // R.operator+(R1);
```

```
    R * R1; // R.operator*(R2);
```

```
    return 0;
```

```
}
```

5. WAP TO SHOW ALL CONVERSION.

```
#include <iostream>

class Temp {
    int A;
    int B;
public:
    Temp (inta=0, intb=0): A(a), B(b) {}
};

class Rect {
    int L;
    int B;
public:
    Rect (inta=0, intb=0): L(a), B(b) {}
    Rect (Temp t): L(T.A), B(T.B) {}
    inline int GetL() { return L; }
    inline int GetB() { return B; }

    void Display() {
        std::cout << "Length = " << L << std::endl;
        std::cout << "Breadth = " << B << std::endl;
    }
};

int main() {
    int a=10, b=20, c, d;
    int e=30, f=15;

    Temp T(e,f);
    Rect r(a,b); // From basic to class datatype.
    Rect R(T); // From one class type to another.
    c = R.GetL(); // For class type to basic type.
    d = R.GetB();
    r.Display();
    R.Display();
    std::cout << "class to basic type. L=" << c << " breadth=" << d << std::endl;
    return 0;
}
```

6. Create a base class called 'Employee' with name, employeeId, and a method 'showInfo()'.
Now create two derive class. (a) manager, with teamsize, and method 'showManagerInfo()' (b) Engineer with specialization and method 'showEngineerInfo()'.
→ In (main()) . create object of manger and employee and enter info and output info.

Sol'n

```
#include <iostream>
#include <string>
class employee{
protected:
    std::string name;
    int eID;
public:
    employee(): name(""), eID(0) {}
    void showEmployeeInfo(){
        std::cout << " \n Employee name = " << name << std::endl;
        std::cout << " \n Employee ID = " << eID << std::endl;
    }
    void SetEmployeeInfo(){
        std::cout << " Enter Ename and Eid = " << std::endl;
        std::cin >> name >> eID;
    }
};

class manager: public employee{
protected:
    int teamSize;
public:
    manager(): name(""), eID(0), teamSize(0) {}
    void showManagerInfo(){
        std::cout << " \n Teamsize = " << teamSize << std::endl;
    }
    void SetteamSize(){
        std::cout << " set teamsize = " << std::endl;
        std::cin >> teamSize;
    }
};
```

```

class Engineer: public employee {
protected:
    std::string specialization;
public:
    Engineer(): name(""), eID(0), specialization("") {}
    void SetSpecialization() {
        std::cout << "Enter Specialization = " << std::endl;
        std::cin >> specialization;
    }
    void ShowEngineerInfo() {
        std::cout << "Specialization = " << specialization << std::endl;
    }
};

```

```

int main() {
    manager m;
    Engineer E;
    m.SetEmployeeInfo();
    m.SetTeamSize();
    E.SetEmployeeInfo();
    E.SetSpecialization();

    m.ShowEmployeeInfo();
    m.ShowManagerInfo();
    E.ShowEmployeeInfo();
    E.ShowEngineerInfo();
    return 0;
}

```

7. WAP TO GIVE EXAMPLE OF POLYMORPHISM OF CLASS SHAPE, RECTANGLE AND TRIANGLE:

```
#include <iostream>
```

```
class Shape {
```

```
protected:
```

```
float L;
```

```
float B;
```

```
public:
```

```
Shape(): L(0), B(0) {}
```

```
virtual void GetDimension() = 0;
```

```
virtual void GiveArea() = 0;
```

```
};
```

```
class Rectangle : public Shape {
```

```
public:
```

```
void GetDimension() override {
```

```
std::cout << "Enter length and breadth = " << std::endl;
```

```
std::cin >> L << B;
```

```
}
```

```
void GiveArea() override {
```

```
float A = L * B;
```

```
std::cout << "Rectangle Area = " << A << std::endl;
```

```
}
```

```
};
```

```
class Triangle : public Shape {
```

```
public:
```

```
void GetDimension() override {
```

```
std::cout << "Enter height and base = " << std::endl;
```

```
std::cin >> L << B;
```

```
}
```

```
void GiveArea() override {
```

```
float A = L * B / 2.0f;
```

```
std::cout << "Area of Triangle = " << A << std::endl;
```

```
}
```

```
};
```

```
int main() {  
    Rectangle R;  
    Triangle T;  
    R.GetDimension();  
    R.Give-Area();  
    T.GetDimension();  
    T.Give-Area();  
    return 0;  
}
```

8. WAP TO HANDLE EXCEPTION FOR INVALID AGE INPUT.

```
#include <iostream>
```

```
#include <stdexcept>
```

```
int divide (int a, int b) {
```

```
    if (b == 0 {
```

```
        throw std::runtime_error ("Division by zero");
```

```
    }  
    else { return a/b;
```

```
    }
```

```
int main() {
```

```
    try {
```

```
        int result = divide (10, 0);
```

```
        std::cout << "Result = " << result << std::endl;
```

```
    } catch (const std::runtime_error & e) {
```

```
        std::cout << "Caught an exception = " << e.what() << std::endl;
```

```
    }
```

```
    return 0;
```

```
}
```

9. WAP TO USE TEMPLATE T AND U TO FIND SMALLER NUMBER BETWEEN T AND U.

```
#include <iostream>
```

```
template < typename T, typename U >
```

```
T getSmaller (Ta, Ub) {
```

```
    return (a < b) ? a : b;
```

```
}
```

```
int main() {
```

```
    int x = 10;
```

```
    double y = 15.5;
```

```
    float z = 8.3f;
```

```
    std::cout << "Smaller between " << x << " and " << y  
                << " is : " << getSmaller(x, y) << std::endl;
```

```
    std::cout << "Smaller between " << y << " and " << z  
                << " is : " << getSmaller(y, z) << std::endl;
```

```
    std::cout << "Smaller between " << x << " and " << z  
                << " is : " << getSmaller(x, z) << std::endl;
```

```
    return 0;
```

```
}
```

10. WAD TO READ A FILE "Data.txt" and DISPLAY ALL THE CONTENT.

```
#include <iostream>
#include <fstream>

int main() {
    std::ifstream file("data.txt");
    if (!file.isopen()) {
        std::cerr << "couldn't open the file" << std::endl;
        return 1;
    }
    char ch;
    while (file.get(ch)) {
        std::cout << ch;
    }
    file.close();
}
```

11. WAP TO DEMONSTRATE RUN TIME POLYMORPHISM IN CLASS ANIMAL BY CLASS BIRD AND FISH.

```
#include <iostream>
```

```
class Animal {
```

```
public:
```

```
virtual void makeSound() {
```

```
    std::cout << "Animal makes a sound" << std::endl; }
```

```
virtual void move() {
```

```
    std::cout << "Animal moves" << std::endl; }
```

```
};
```

```
class Bird: public Animal {
```

```
public:
```

```
void makeSound() override {
```

```
    std::cout << "Bird sings" << std::endl; }
```

```
void move() override {
```

```
    std::cout << "Bird flies" << std::endl; }
```

```
};
```

```
class Fish: public Animal {
```

```
public:
```

```
void makeSound() override {
```

```
    std::cout << "Fish make bubbles" << std::endl; }
```

```
void move() override {
```

```
    std::cout << "Fish swims" << std::endl; }
```

```
};
```

```
int main() {
```

```
    Animal animal; // create a default animal
```

```
    Bird bird; // create a bird
```

```
    Fish fish; // create a fish
```

```
    animal.makeSound(); // output Animal makes a sound
```

```
    animal.move(); // output Animal moves
```

```
    bird.makeSound(); // output Bird sings
```

```
    bird.move(); // output Bird flies
```

```
    fish.makeSound(); // output Fish make bubbles
```

```
    fish.move(); // output Fish swims
```

```
    return 0;
```

```
}
```

12. WAP TO HANDLE EXCEPTION THROWN WHEN DIVIDING BY NULL

```
#include <iostream>
#include <stdexcept>
```

```
void divide (int N, int D) {
```

```
    if (D==0) { throw std::runtime_error("division by 0"); }
```

```
    else { std::cout << "Result = " << static_cast<float>(N)/D << std::endl; }
}
```

```
}
```

```
int main () {
```

```
    int n1=10, n2=20, n3=30, n4=0;
```

```
    try {
```

```
        divide (n2, n1);
```

```
    } catch (std::runtime_error e) {
```

```
        std::cerr << "caught an exception = " << e.what() << std::endl;
```

```
    }
```

```
    try { divide (n3, n4); }
```

```
    catch (std::runtime_error e) {
```

```
        std::cerr << "caught an exception = " << e.what() << std::endl;
```

```
    }
```

```
    return 0;
```

```
}
```

13 WAP TO CREATE A CLASS TEMPLATE 'pair' WHICH STORE TWO DIFFERENT DATATYPES.

```
#include <iostream>
#include <string>
template < typename T1, typename T2 >
class Pair {
public:
    Pair(const T1 & F, const T2 & S) : First(F), Second(S) {}
    void First(T1 & T) { First = T; }
    void Second(T2 & T) { Second = T; }
    void Display() {
        std::cout << "First = " << First << ", " << "Second = " << Second
            << std::endl;
    }
    inline void SFirst() { return First; }
    inline void SSecond() { return Second; }
private:
    T1 First; T2 Second;
};

int main() {
    Pair<int, double> pair1(1, 3.14);
    pair1.Display(); // First = 1, Second = 3.14 output
    pair1.First(10);
    pair1.Second(2.78);
    pair1.Display();

    Pair<std::string, char> pair2("Hello", 'A');
    pair2.Display();
    pair2.First("World");
    pair2.Second('B');
    pair2.Display();
    return 0;
}
```

1) This is a basic version of std::pair of C++ std Library

14

WAP TO COPY DETAIL OF EMPLOYEE FROM "Source.txt" TO "Dest.txt" File
DETAIL INCLUDE EID, NAME, POSITION AND SALARY.

```
#include <fstream>
#include <iostream>
#include <string>

int main() {
    File src("Source.txt"); std::istream src("Source.txt");
    File dst("Dest.txt"); std::ostream dst("Dest.txt");
    int EId; float salary;
    std::string name, position;
    while (src >> EId >> name >> position >> salary) {
        dst << EId << " " << name << " " << position << " " << salary
        << std::endl;
    }
    src.close(); dst.close();
    return 0;
}
```

15

WAP TO DISPLAY DETAIL OF EMPLOYEE. DETAIL INCLUDE EID,
NAME, POSITION AND SALARY.

```
#include <string>
#include <iostream>
#include <fstream>

int main() {
    std::istream file("data.txt");
    int eid;
    float salary;
    std::string name, position;
    while (file >> eid >> name >> position >> salary) {
        std::cout << "id = " << eid << ", " << "name = " << name << ", "
        << "position = " << position << ", " << "salary = " << salary
        << std::endl;
    }
    file.close();
    return 0;
}
```